
Web Programming with Python and Git

- **Yüksel Arslan**

Asst.Prof. Yüksel Arslan

Ankara Science University

Assessment Methods

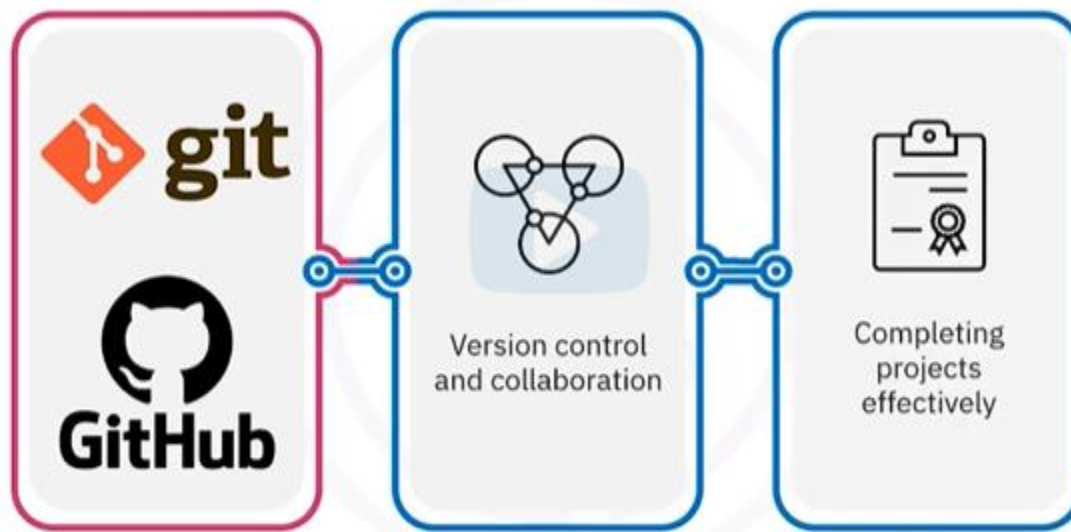
	Type	Count	Total Contribution
1	Final: Essay/written	1	30
2	Midterm: Essay/written	1	30
3	Project	1	30
	Attendance	14	10

Note: It may change

Git and GitHub

Version control and Collaboration

- **Version control** and **collaboration software** are crucial for completing software development projects effectively.



What is version control?

- a Version Control System :



Tracks changes to
source code



Recovers older versions



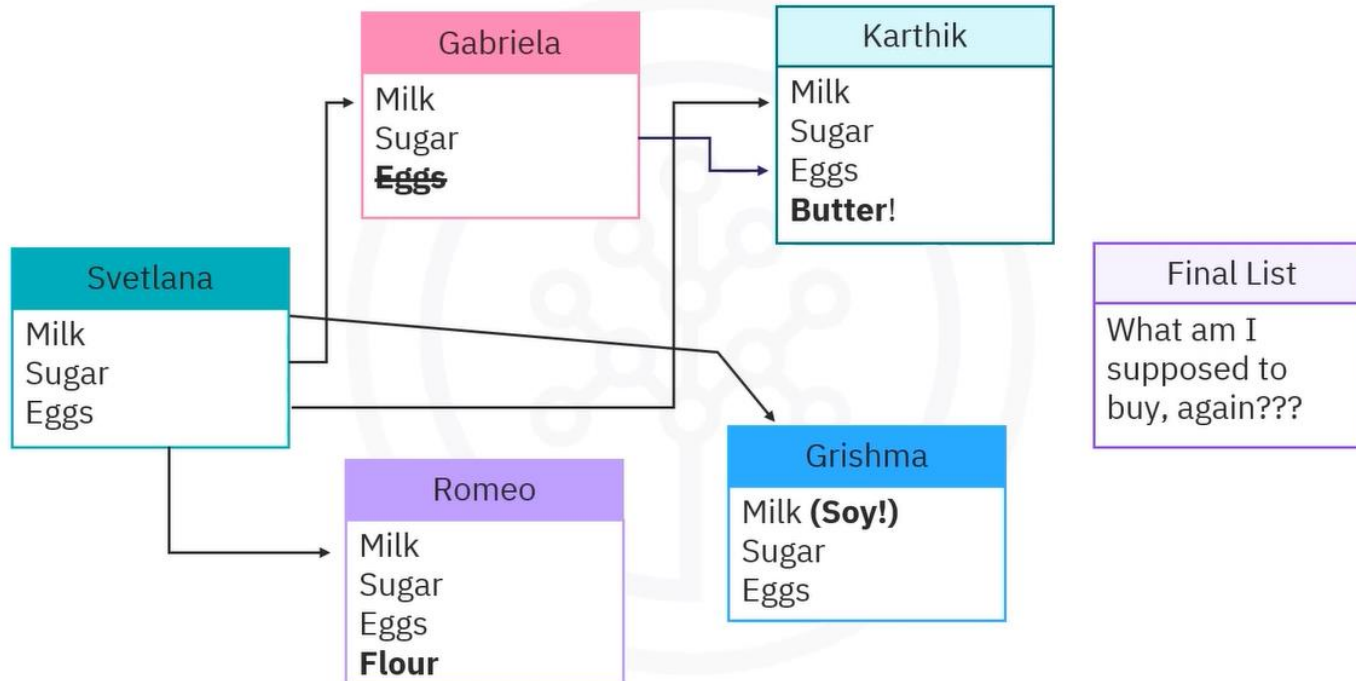
Facilitates collaboration

What is version control?



What is version control?

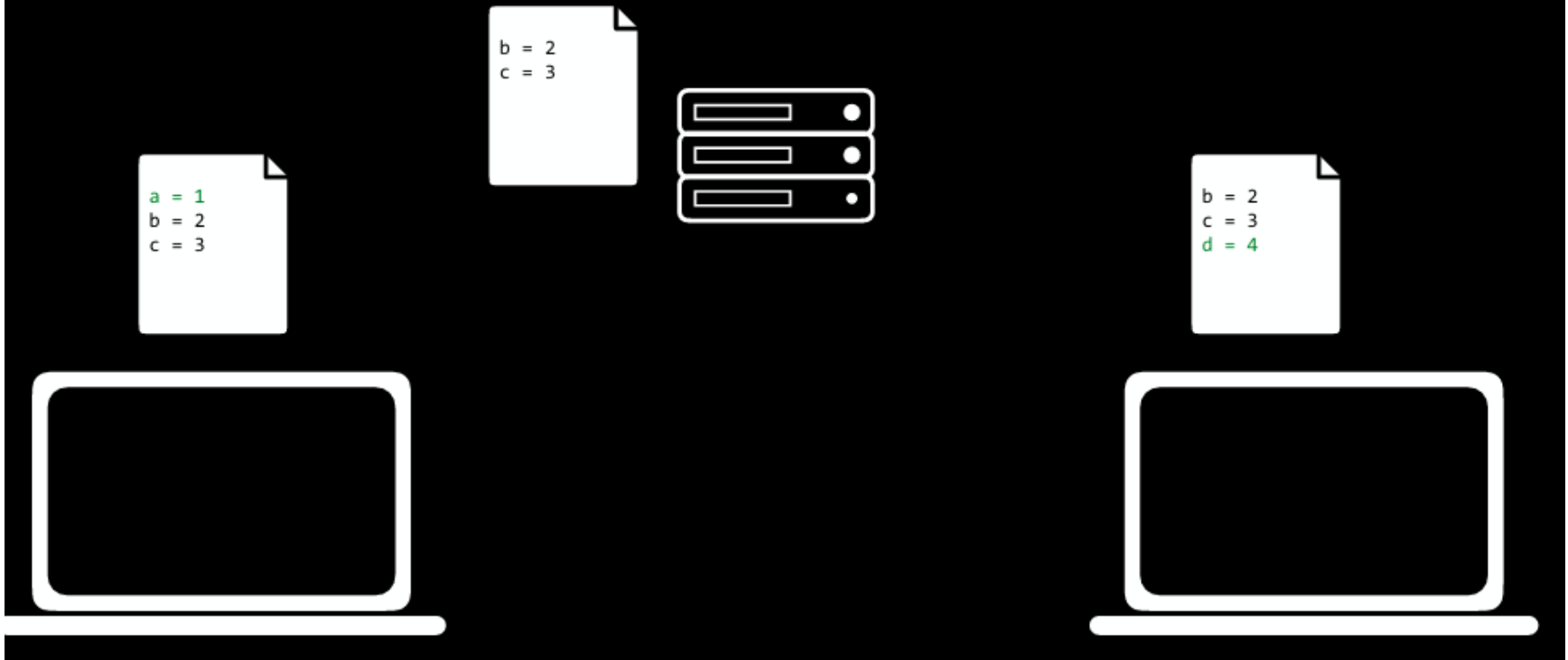
Working without version control



Let's say you have a shopping list and you want your roommates to confirm the things you need and add additional items. Without version control, you've got a big mess to clean up before you can go shopping, with version control you know exactly what you need after everyone has contributed their ideas.

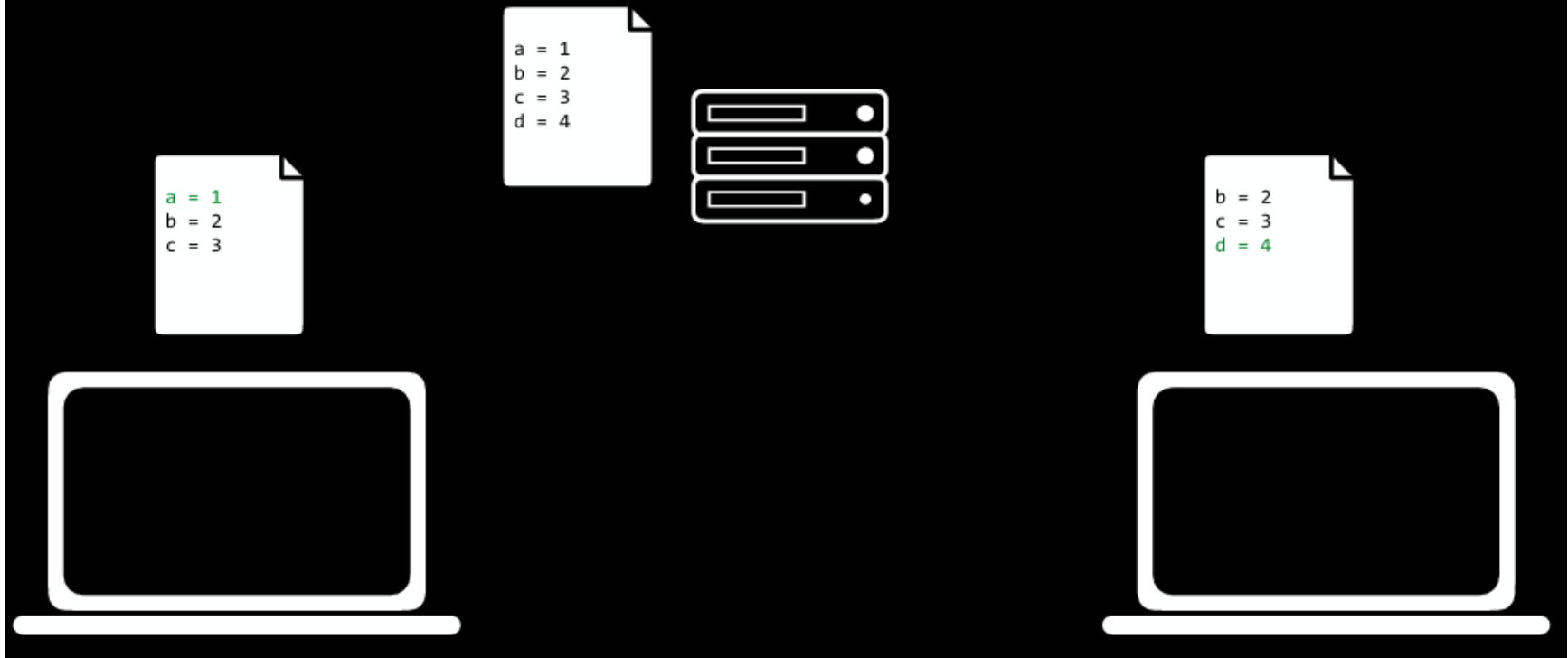
What is version control?

Synchronizes code between different people.



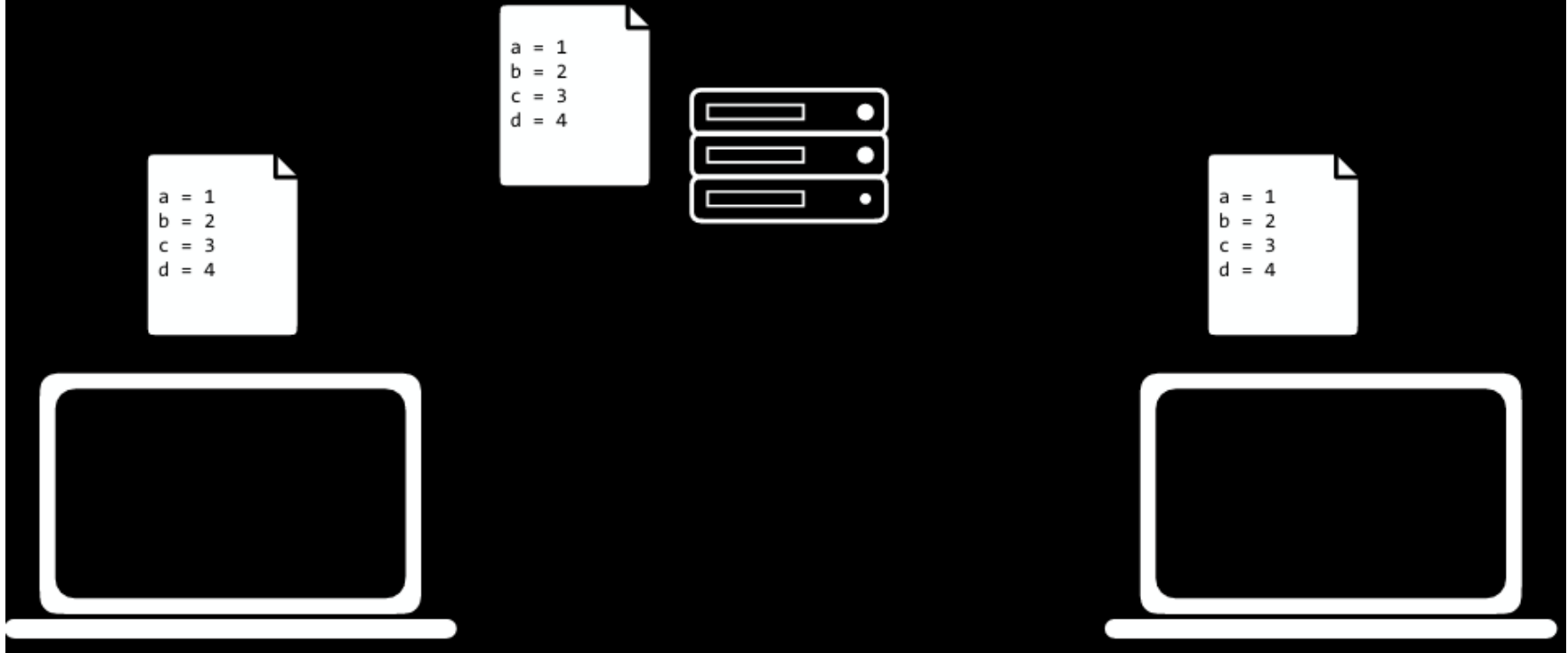
What is version control?

Synchronizes code between different people.



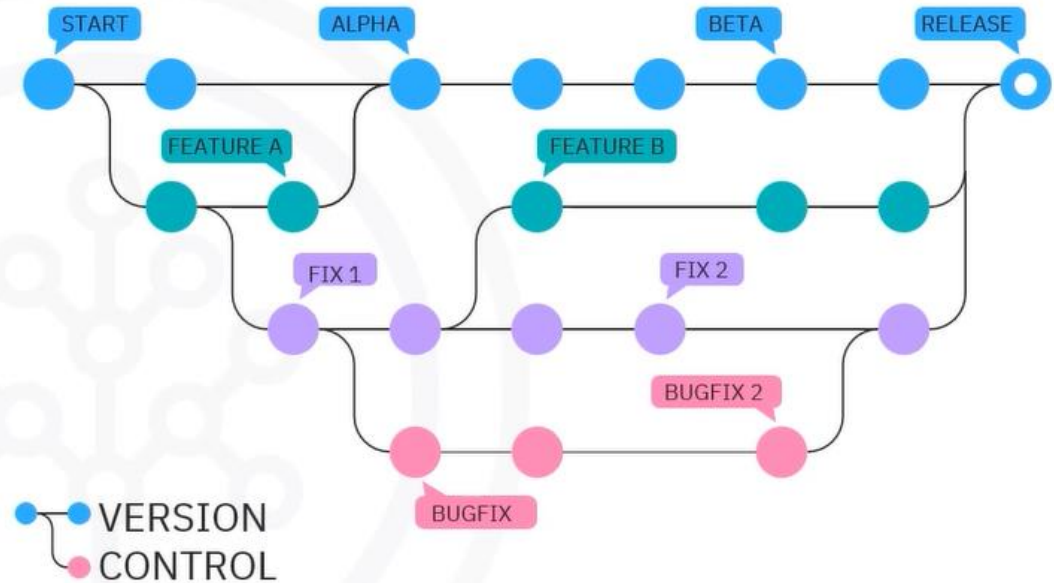
What is version control?

Synchronizes code between different people.



Git and GitHub

- Free and open-source software under GNU
- DVCS: Easy accessibility of project copies



Git, is a distributed version control system, or DVCS, meaning users anywhere in the world can have a copy of your project on their computer. When they've made changes, they can sync their version to a remote server to share it with you.

What is version control?

- a **Version Control System** is just software that helps you control (or manage) the different versions of something (typically **source code**).
- the main point of a version control system is to help you maintain a **detailed history of the project** as well as **the ability to work on different versions of it**.
Having a detailed history of a project is important because it lets you see the progress of the project over time. If needed, **you can also jump back to any point in the project to recover data or files**.

Categories of Version Control Systems

Git

Subversion

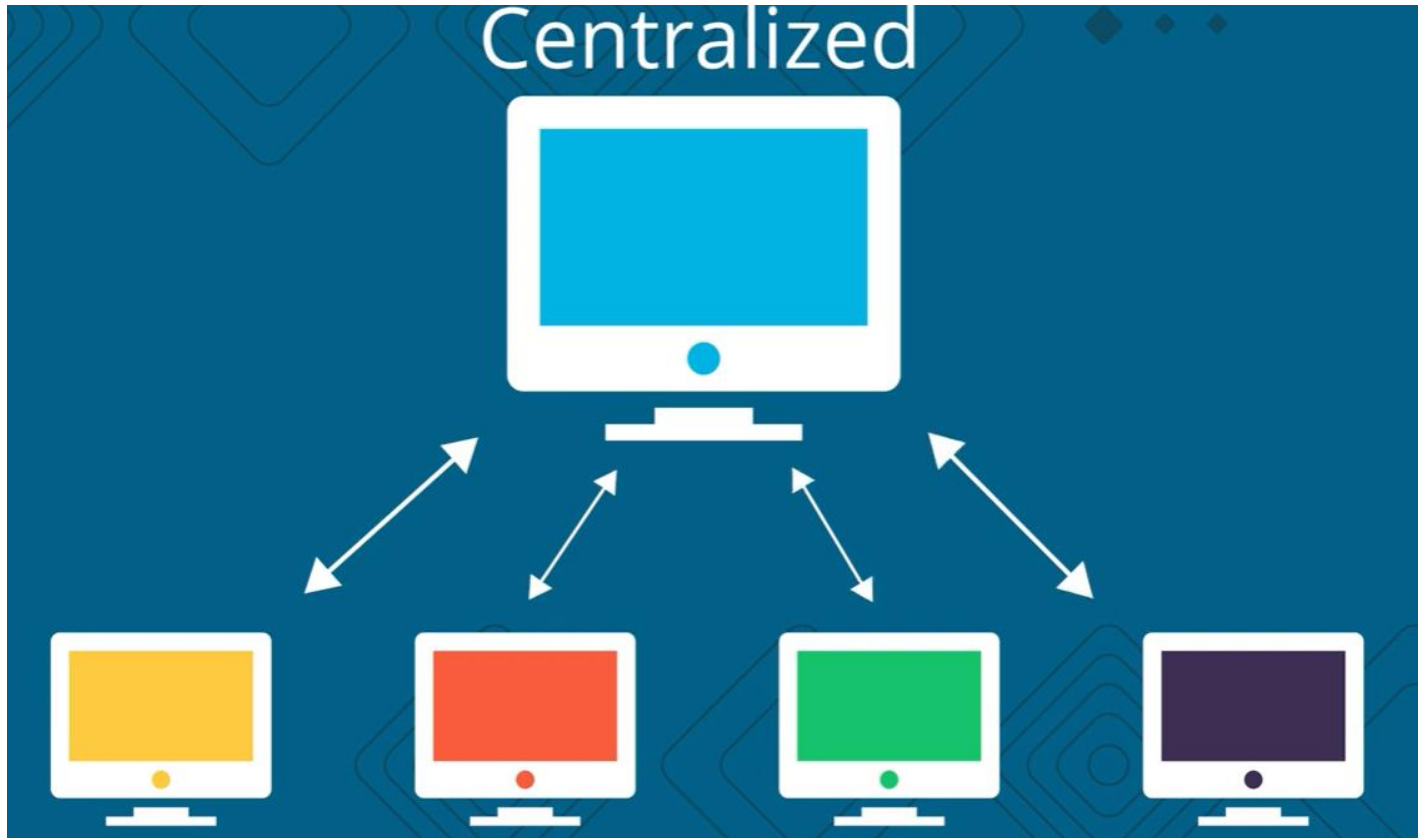
Mercurial

Centralized

Distributed

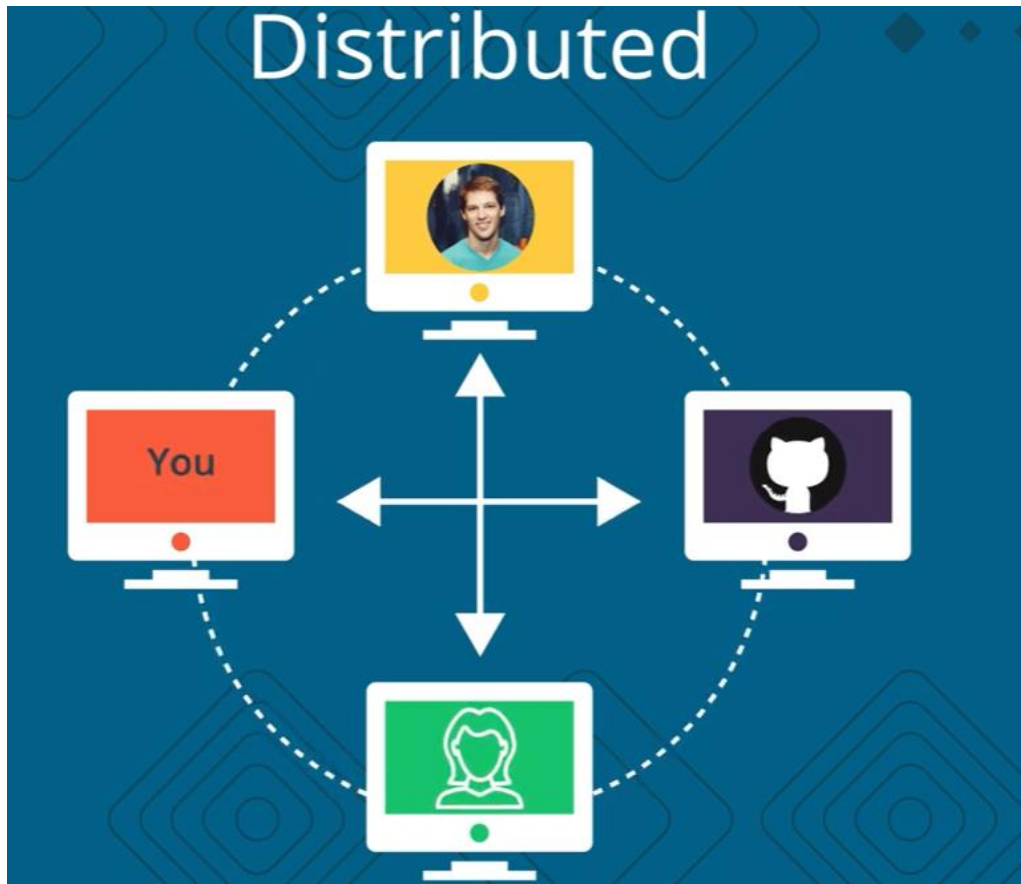
Categories of Version Control Systems

Centralized: the centralized model - all users connect to a central, master repository.



Categories of Version Control Systems

Distributed: the distributed model - each user has the entire repository on their computer. It means you can work offline.



Categories of Version Control Systems



Categories of Version Control Systems



Version control tool



Service that hosts
Git projects

Defining Version Control

- Terminology:



SSH protocol: Method to secure remote login



Repository: Contains project folders



Fork: Copy of a repository



Pull request: To request for review and approval



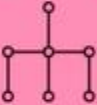
Working directory: Contains files and subdirectories

Defining Version Control

- Terminology:



Commit: Snapshot of the project's current state



Branch: Separate line of development



Merge: Combines changes from one branch to another



Clone: Local copy of the remote Git repository

Defining Version Control

- Terminology:

- **A version control system** (abbreviated as **VCS**) is a tool that manages different versions of source code. A source code manager (abbreviated as **SCM**) is another name for a version control system.
- Git is an SCM (and therefore a VCS!). The URL for the Git website is <https://git-scm.com/> (see how it has "SCM" directly in its domain!).

Defining Version Control

- Terminology:

- **Commit:**

- Git thinks of its data like a set of snapshots of a mini filesystem. Every time you commit (save the state of your project in Git), it basically takes a picture of what all your files look like at that moment and stores a reference to that snapshot. You can think of it as a save point in a game - it saves your project's files and any information about them.

- **Repository / repo :**

- A repository is a directory which contains your project work, as well as a few files which are used to communicate with Git. Repositories can exist either locally on your computer or as a remote copy on another computer. A repository is made up of commits.

Defining Version Control

- Terminology:

- **Working Directory:**

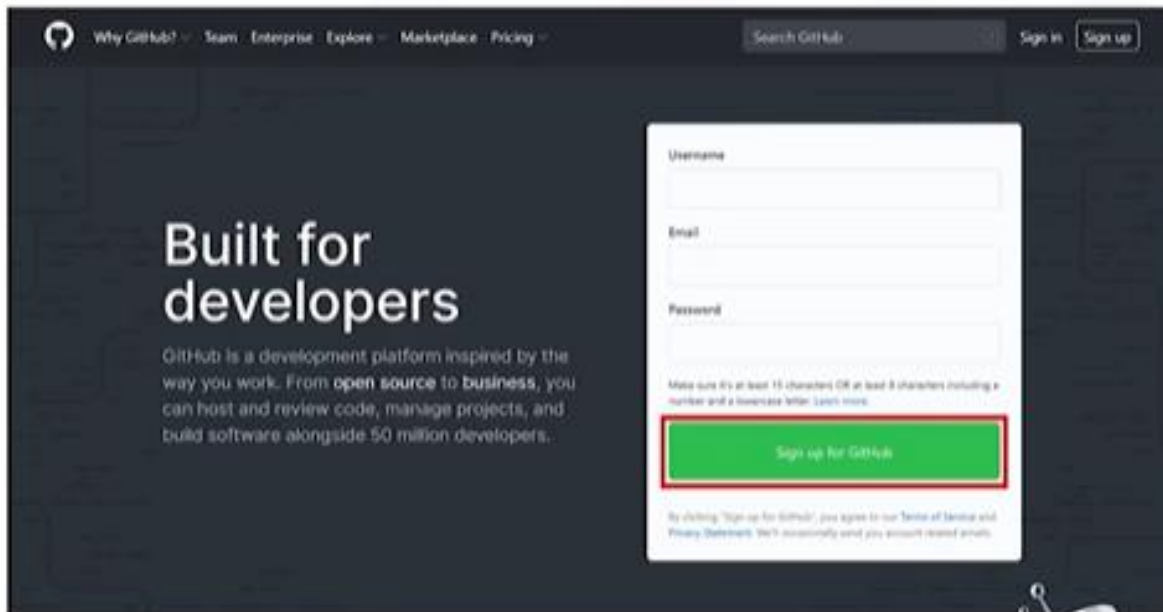
- The Working Directory is the files that you see in your computer's file system. When you open your project files up on a code editor, you're working with files in the Working Directory.
 - This is in contrast to the files that have been saved (in commits!) in the repository.
 - When working with Git, the Working Directory is also different from the command line's concept of the current working directory which is the directory that your shell is "looking at" right now.

- **Staging Area / Staging Index / Index:**

- A file in the Git directory that stores information about what will go into your next commit. You can think of the staging area as a prep table where Git will take the next commit. Files on the Staging Index are poised to be added to the repository.

Create a GitHub Account

Create an Account

A screenshot of the GitHub website's sign-up page. The page has a dark background with white text. On the left, it says "Built for developers" and describes GitHub as a development platform. On the right, there is a white sign-up form with fields for "Username", "Email", and "Password". Below the password field, there is a green button labeled "Sign up for GitHub" which is highlighted with a red rectangular border. Above the button, there is a small note about password requirements. At the bottom of the form, there is a line of small text about agreeing to terms and privacy.

Why GitHub? Team Enterprise Explore Marketplace Pricing Search GitHub Sign in Sign up

Built for developers

GitHub is a development platform inspired by the way you work. From open source to business, you can host and review code, manage projects, and build software alongside 50 million developers.

Username

Email

Password

Make sure it's at least 10 characters OR at least 8 characters including a number and a lowercase letter. Learn more.

Sign up for GitHub

By clicking "Sign up for GitHub", you agree to our Terms of Service and Privacy Statement. We'll occasionally send you account related emails.

- Go to <https://github.com>
- Choose a username
- Enter your email address
- Create a password
- Click Sign up for GitHub



Install git

- Installing Git
 - To download Git:
 - go to **<https://git-scm.com/downloads>**(opens in a new tab)
 - download the software for Windows
 - install Git choosing all of the default options
 - Once everything is installed, you should be able to run git on the command line. If it displays the usage information, then you're good to go!
 - Check git installed correctly
 - `git --version`

Install git

First Time Git Configuration

Before you can start using Git, you need to configure it. Run each of the following lines on the command line to make sure everything is set up.

sets up Git with your name

```
git config --global user.name "<Your-Full-Name>"
```

sets up Git with your email

```
git config --global user.email "<your-email-address>"
```

makes sure that Git output is colored

```
git config --global color.ui auto
```

displays the original state in a conflict

```
git config --global merge.conflictstyle diff3
```

```
git config --list
```

VSCode Setup

```
git config --global core.editor "code --wait"
```

Create a Repository

Key Terms/Commands

1. **Repo**: the shorthand version of the term, repository
2. **git init** : create brand new repos
3. **git clone**: clone or copy an existing repo
4. **git status**: check the status of a repo

Create a Repository

- Create a repo on the GitHub
 - Using web interface commands after you login to your GitHub account
 - To use it we must make a clone of it in the
- Create a repo on local computer
 - Use git init command
 - Then you must link it with a remote repo

Create a Repository on GitHub

- The heart of a Git-based project is the **repository**.
- **This contains all your code and the related artifacts**, including things like:
 - A README file to describe the purpose of the project.
 - A license to express the ways in which people can use your code, Etc.
- You can make your repository **private** (only available to people with accounts that have permission to see it) or **public** (searchable and seen by everyone).

Defining Version Control

- You have some files in a directory, this directory may be called repository, but it is not a git repository.



Defining Version Control

- We want to start using version control in this project. So we create a new git repository. In Git repo there are three distinct areas that we need to know about.

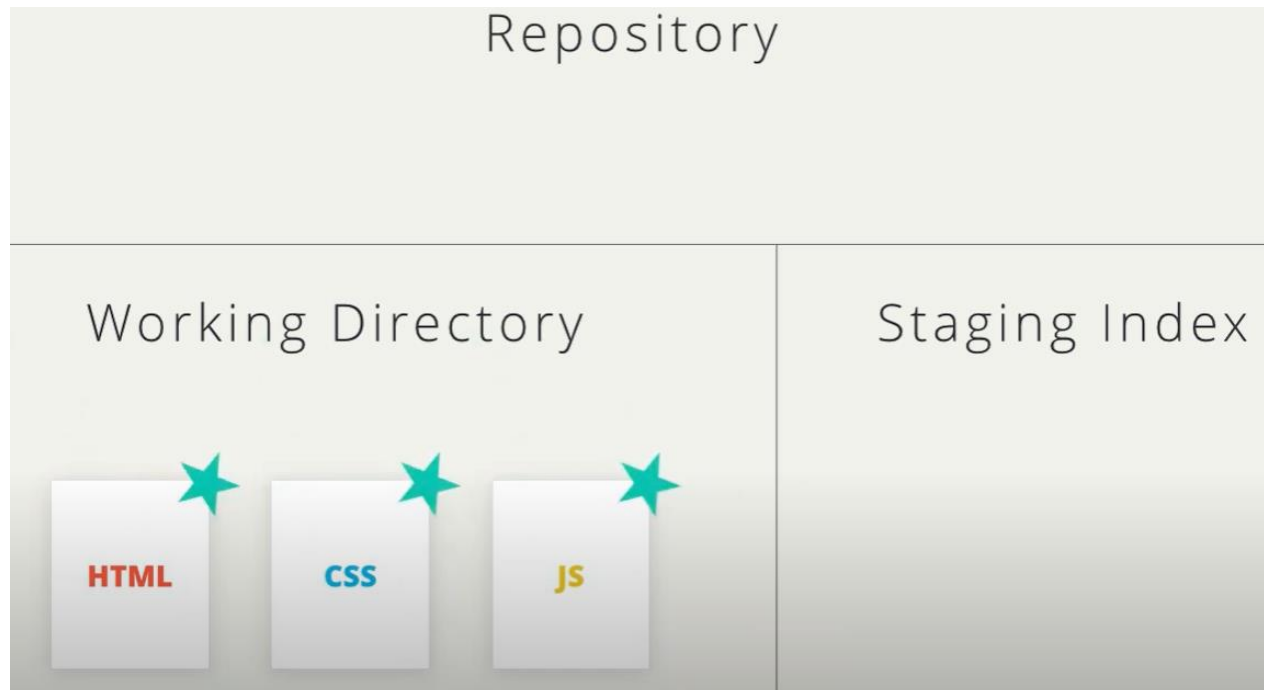


Defining Version Control

- It is hard to realize these three distinct areas, because you can not see any visible change on your computer.
- Git manages these areas.
- And we use **Git shell commands** to **to interact with Git** and **to move files from section to section**.
- As you see at the beginning the Git repository is empty.

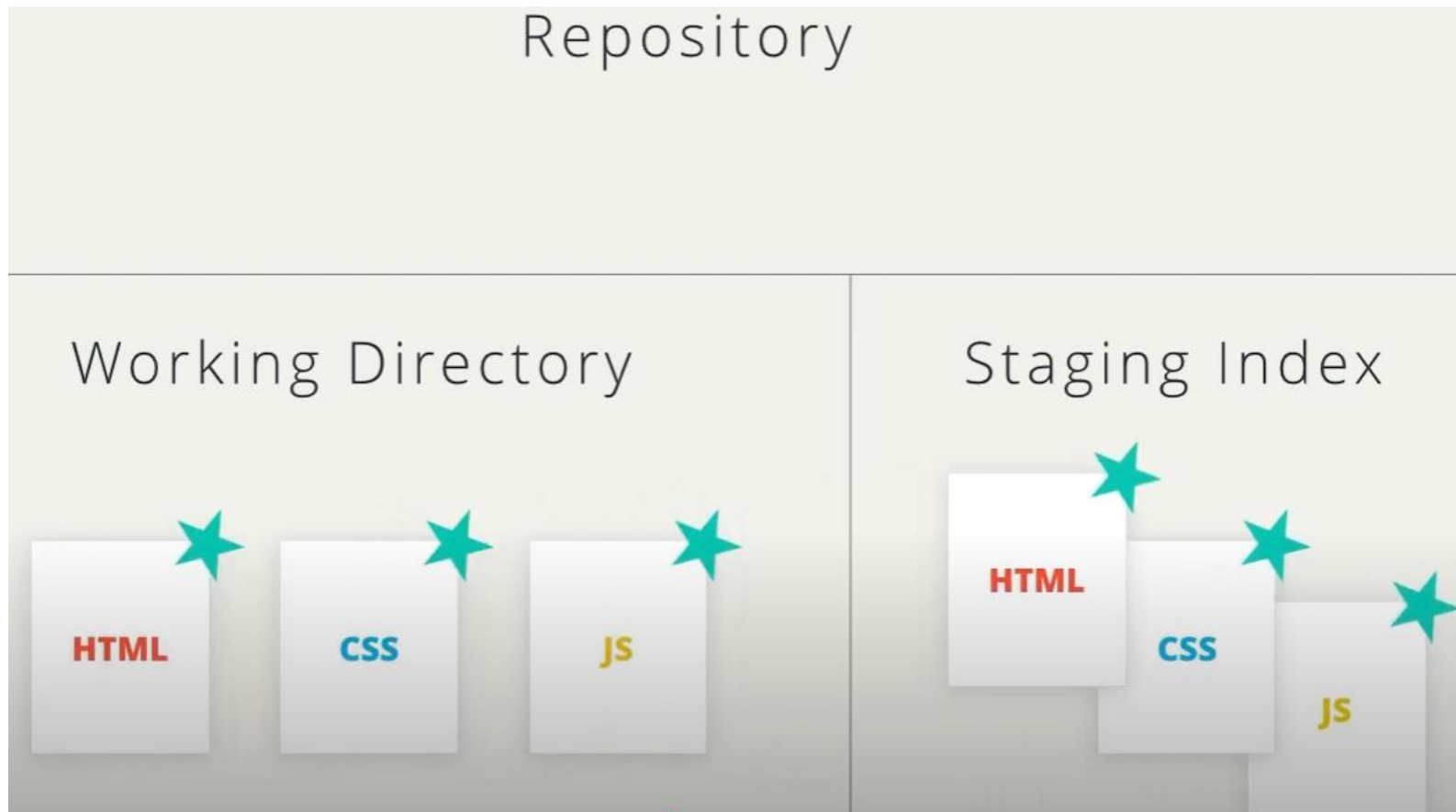
Defining Version Control

- As you see at the beginning the Git repository section is empty.
- **Git knows about files that it hasn't seen before.** Let's indicate new files with a green star.
- Git keep tracks of files by adding them to the repo.



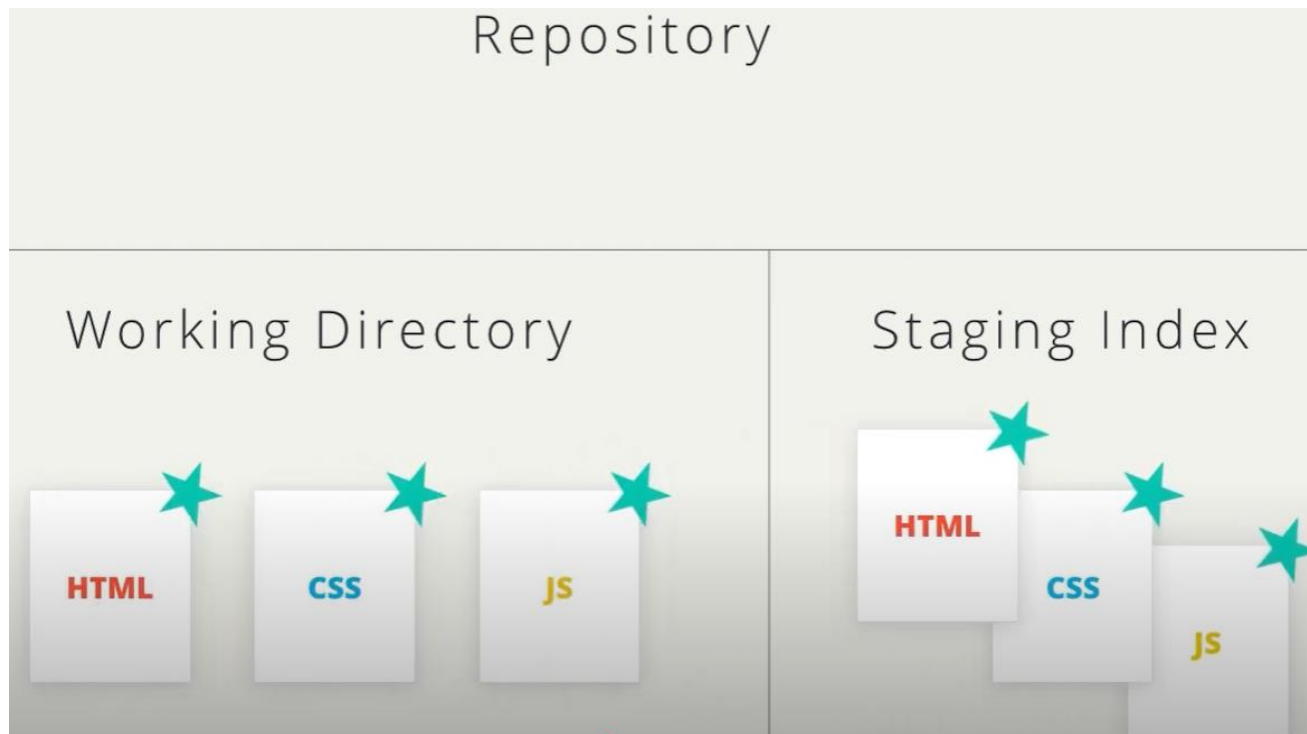
Defining Version Control

- We can not move files directly to repo, by using Git command first we must move them staging index first



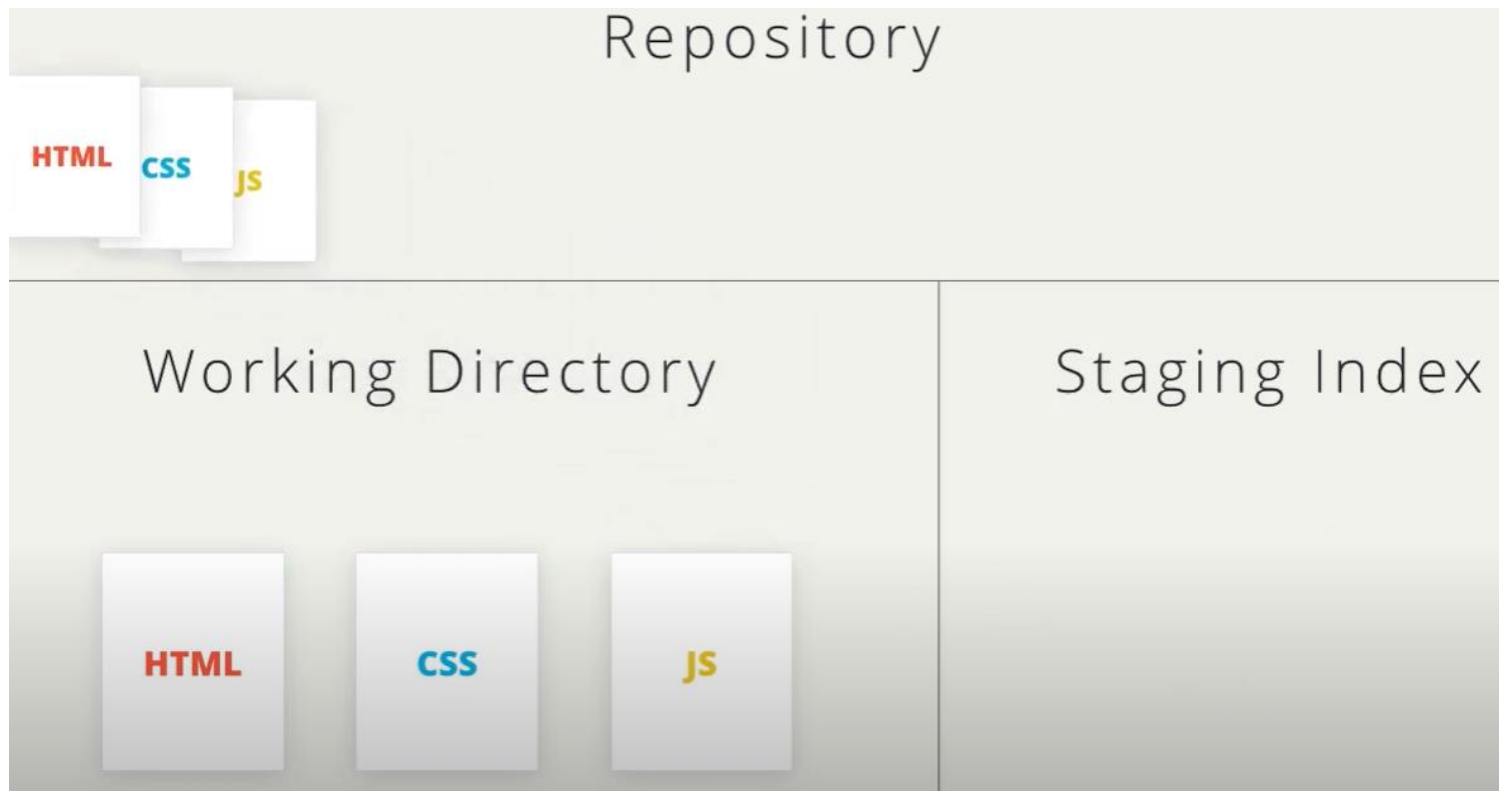
Defining Version Control

- We can not move files directly to repo, by using Git command first we must move them staging index first.
- Staging index holds the files that are changed or new to the repo.
- Staging index is a place where we put files that are about to be committed.



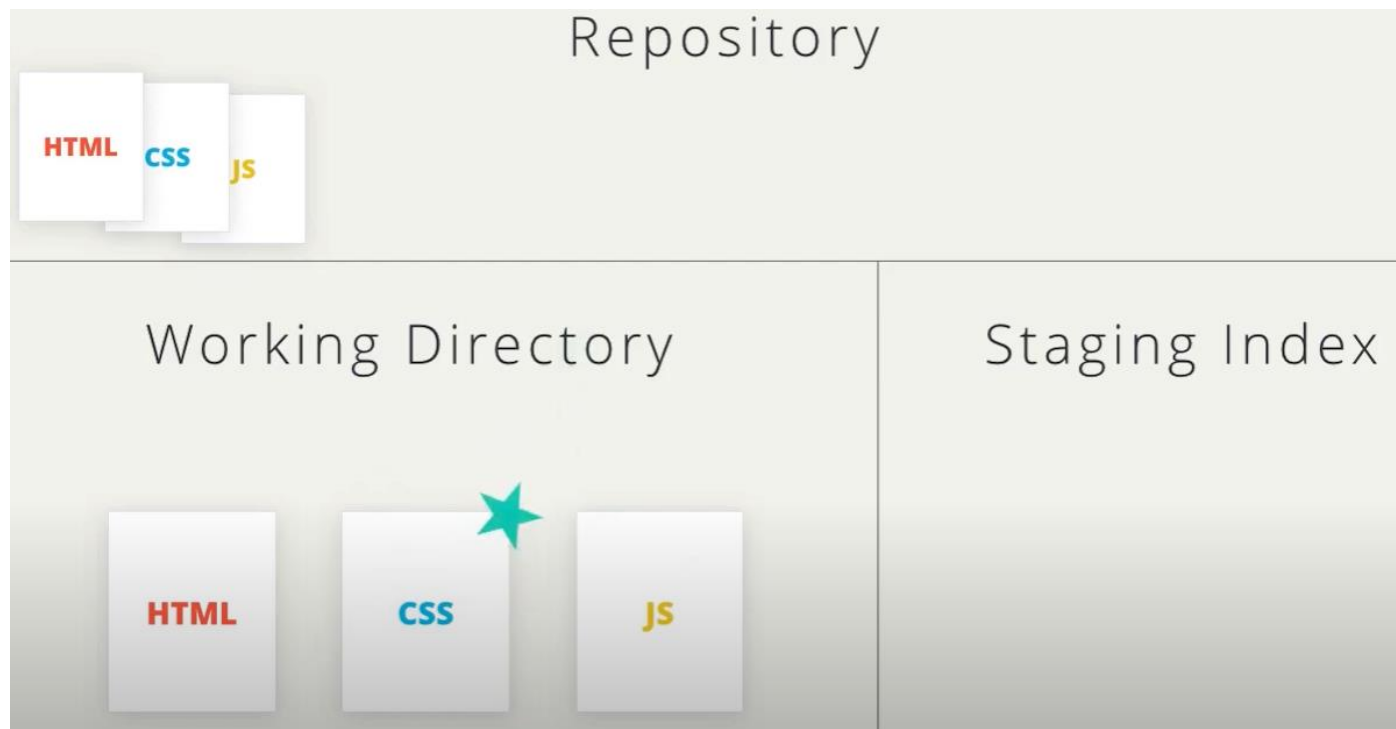
Defining Version Control

- And at last, we commit them, the files are moved to repository section. And now their current state is safe now.
- Note that there are no green stars on the files, and the staging index is empty.



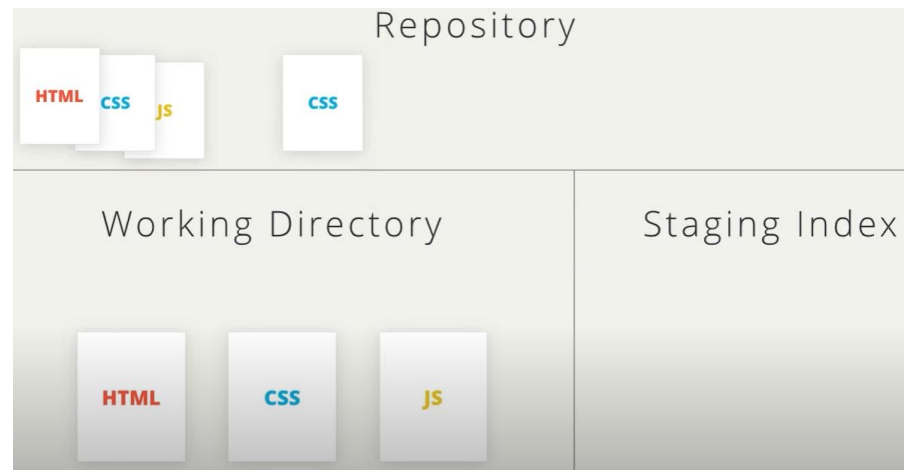
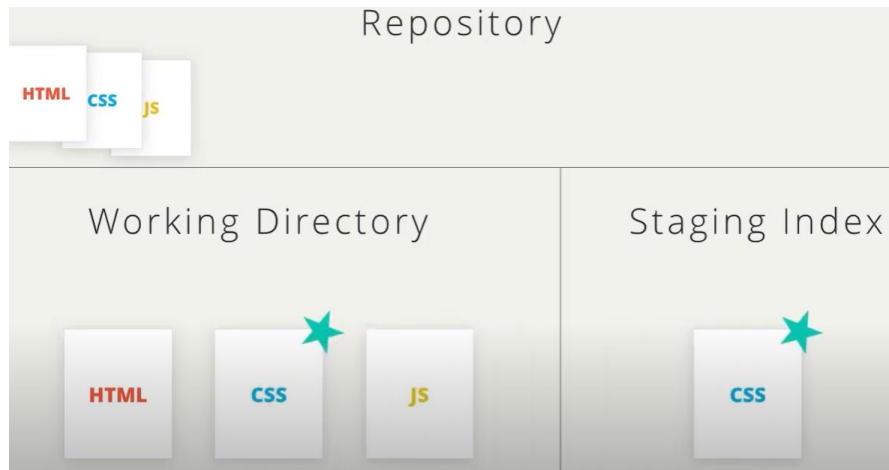
Defining Version Control

- Let's make a change to our CSS file. **Git sees this change** and mark it as modified.
- We take the same steps to move the changes to staging index and commit them



Defining Version Control

- Let's make a change to our CSS file. Git sees this change and mark it as modified.
- We take the same steps to move the changes to staging index and commit them



Defining Version Control

The overall process is:

- Add a new file or modify existing one, Git keep tracks of these files automatically
- Move changes to the staging index
- Commit the changes to the repository.

Defining Version Control

Question:

- Let's say HTML and JS files have changed and we staged these changes.
- Then we decide to modify HTML file again,
- At this point what would be committed if we made a commit right now

only the HTML file changes on the Staging Index

only the CSS file changes on the Staging Index

the HTML and CSS file changes on the Staging Index

the HTML file changes in *both* the Working Directory *and* the Staging Index

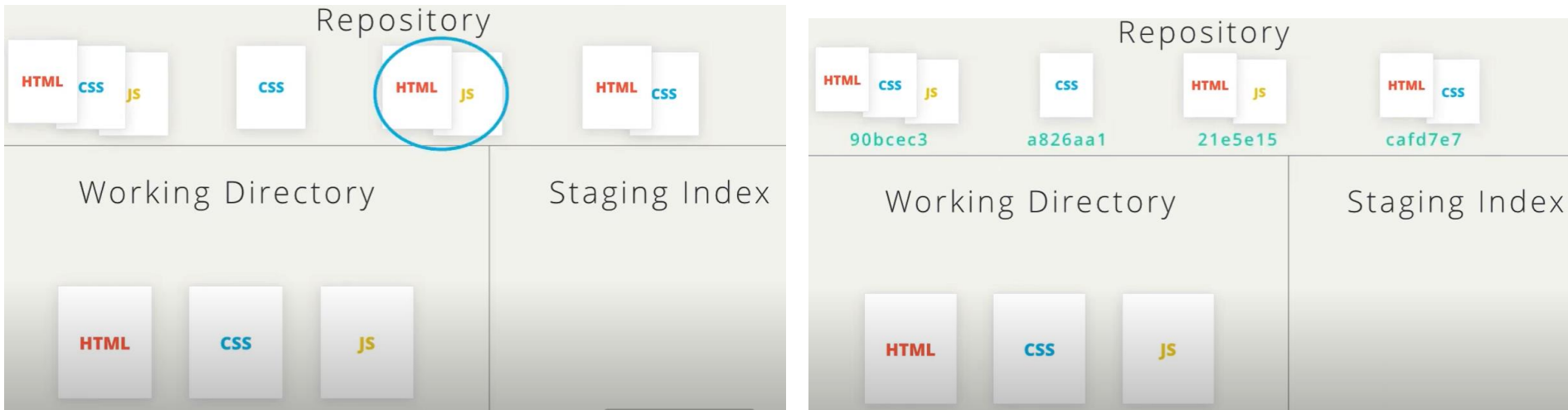
Defining Version Control

Question:



Defining Version Control

How can we access each commit?



When each commit is made, Git creates an ID for it.

The ID for the commit is its SHA, you see seven characters of each commit's SHA

Defining Version Control

- More Terminology:

- Branch:

- A branch is when a new line of development is created that diverges from the main line of development. This alternative line of development can continue without altering the main line.
 - Going back to the example of save point in a game, you can think of a branch as where you make a save point in your game and then decide to try out a risky move in the game. If the risky move doesn't pan out, then you can just go back to the save point. The key thing that makes branches incredibly powerful is that you can make save points on one branch, and then switch to a different branch and make save points there, too.
 - With this terminology in mind, let's take a high-level look at how we'll be using Git by looking at the typical workflow when working with version control.

Defining Version Control

- **More Terminology:**

- **Checkout**

- A checkout is when content in the repository has been copied to the Working Directory.

- **SHA**

- A SHA is basically an ID number for each commit. Here's what a commit's SHA might look like: e2adf8ae3e2e4ed40add75cc44cf9d0a869afeb6.
 - It is a 40-character string composed of characters (0–9 and a–f) and calculated based on the contents of a file or directory structure in Git. "SHA" is shorthand for "Secure Hash Algorithm".

Install Git

- **Installing Git**
- To download Git:
 - go to <https://git-scm.com/downloads> (opens in a new tab)
 - download the software for Windows
 - install Git choosing all of the default options
 - Once everything is installed, you should be able to run git on the command line. If it displays the usage information, then you're good to go!

Create a Git Repo and View Repo History

In this Lecture:

- Create a new repository
- Copy an existing repository
- Review the "state" of a Git repository
- Review a repo's commit history git log
- Customize a git log's output

Create a Git Repo and View Repo History

git init

Create brand new repositories (repos) on your computer

git clone

Copy existing repos from somewhere else to your local computer

git status

Check the status of a repo

Create a Git Repo and View Repo History

- Git init
 - Use the git init command to create a new, empty repository in the current directory.
 - C:> **git** init
 - Running this command creates a hidden .git directory. This .git directory is the brain/storage center for the repository. It holds all of the configuration files and directories and is where all of the commits are stored.